

CSC315 Number System, Codes and Addressing Modes (2)

Condensed Study Notes

Generated: 2026-05-24 23:26 UTC

COMPUTER ARCHITECTURE AND ORGANIZATION AFIT, KADUNA LECTURE NOTE

This document introduces Computer Architecture and Organization.

Key focus areas include:

- Understanding the fundamental design and operational structure of computers.
- Exploring how these components are organized to achieve specific functionalities.

1.0 Number System

Number systems are fundamental to computing. The most commonly used systems are:

- **Binary:** Base 2
- **Hexadecimal:** Base 16
- **Octal:** Base 8
- **Decimal:** Base 10

The base, or radix, of a number system determines the range and quantity of digits it uses. If a system has a base 'r', its digits range from 0 to r-1, meaning there are 'r' unique digits in total. Any base greater than or equal to two can define a valid number system.

1.1 Decimal Number System

The decimal number system is a positional number system that uses ten digits (0-9). It's the system we use daily, likely due to humans having ten fingers.

In a positional system, each digit's value depends on its position. Each position has a weight, which is a power of the base (radix).

For the decimal system, the base (radix) is 10. Positions represent successive powers of 10.

- Positions to the left of the decimal point represent positive powers of 10 (e.g., 10^0 for units, 10^1 for tens, 10^2 for hundreds).
- Positions to the right of the decimal point represent negative powers of 10 (e.g., 10^{-1} for tenths, 10^{-2} for hundredths).

Example: The number 4728 means $(4 \cdot 10^3) + (7 \cdot 10^2) + (2 \cdot 10^1) + (8 \cdot 10^0)$.

Example: The decimal fraction 0.256 means $(2 \cdot 10^{-1}) + (5 \cdot 10^{-2}) + (6 \cdot 10^{-3})$.

1.1.1 The Binary System

The binary system is fundamental to all digital circuits and systems.

- **Base (Radix):** 2
- **Digits Used:** 0 and 1

Binary Point: Divides a binary number into an integer part (left of the point) and a fractional part (right of the point).

Positional Weights:

- **Left of Binary Point (Integer Part):** Successive positions have weights of 2^0 , 2^1 , 2^2 , 2^3 , and so on.
- **Right of Binary Point (Fractional Part):** Successive positions have weights of 2^{-1} , 2^{-2} , 2^{-3} , and so on.

Each position's value is determined by its specific weight, which is a power of the base (2).

Example 1

To convert a binary number to its decimal equivalent, we use mathematical representation where simplified right-hand side terms yield the decimal value.

Consider the binary number 1101.011:

- Integer part: 1101. Digits have positional weights of 2^0 , 2^1 , 2^2 , 2^3 from right to left.
- Fractional part: 0.011. Digits have positional weights of 2^{-1} , 2^{-2} , 2^{-3} from left to right.

a) Decimal Number to other Bases Conversion

To convert a decimal number with both an integer and a fractional part to another base 'r', convert each part separately.

Integer Part Conversion:

1. Divide the integer part of the decimal number by the target base 'r'.
2. Note down the remainder.
3. Repeat the division with the successive quotients.
4. Continue until the quotient becomes zero.
5. The equivalent integer in base 'r' is formed by the remainders, read in reverse order (last remainder is the most significant digit, first remainder is the least significant digit).

Fractional Part Conversion:

1. Multiply the fractional part of the decimal number by the target base 'r'.
2. Note down the carry (the integer part of the result).
3. Repeat the multiplication with the successive fractional parts of the results.

4. Continue until the fractional part becomes zero or the desired number of equivalent digits is obtained.
5. The equivalent fractional part in base 'r' is formed by the carries, read in their normal sequence (first carry is the most significant digit after the radix point).

1.1.2 CONVERTING BETWEEN BINARY AND DECIMAL

This section focuses on the practical steps of converting numbers between the binary (base-2) and decimal (base-10) systems.

Binary to Decimal Conversion:

To convert a binary number to its decimal equivalent, each binary digit (bit) is multiplied by its corresponding power of 2, based on its position relative to the binary point. The sum of these products yields the decimal value.

Example: Convert 101.11 (binary) to decimal.

1. Assign positional weights (powers of 2) starting from the right of the binary point: ... 2^2 , 2^1 , 2^0 . 2^{-1} , 2^{-2}
2. For 101.11: $(1 \cdot 2^2) + (0 \cdot 2^1) + (1 \cdot 2^0) + (1 \cdot 2^{-1}) + (1 \cdot 2^{-2})$
3. Calculate: $(1 \cdot 4) + (0 \cdot 2) + (1 \cdot 1) + (1 \cdot 0.5) + (1 \cdot 0.25) = 4 + 0 + 1 + 0.5 + 0.25 = 5.75$ (decimal).

Decimal to Binary Conversion:

To convert a decimal number to binary, the process differs for the integer and fractional parts.

- **Integer Part:** Repeatedly divide the decimal integer by 2. The remainders, read from bottom to top, form the binary integer.
- **Fractional Part:** Repeatedly multiply the decimal fraction by 2. The integer parts of the results, read from top to bottom, form the binary fraction.

Example: Convert 13.625 (decimal) to binary.

- Integer Part (13):
 - $13 / 2 = 6$ remainder 1
 - $6 / 2 = 3$ remainder 0
 - $3 / 2 = 1$ remainder 1
 - $1 / 2 = 0$ remainder 1
 - Binary integer: 1101
- Fractional Part (0.625):
 - $0.625 \cdot 2 = 1.25$ (integer part 1)
 - $0.25 \cdot 2 = 0.5$ (integer part 0)
 - $0.5 \cdot 2 = 1.0$ (integer part 1)
 - Binary fraction: .101

- Combined: 13.625 (decimal) = 1101.101 (binary).

Example 2

This example demonstrates converting the decimal number 58.25 to its binary equivalent.

Integer Part Conversion (58):

- Repeatedly divide the integer part by 2, recording the quotient and remainder.
- The binary digits are the remainders, read from bottom to top (LSB to MSB).

Operation	Quotient	Remainder
58 / 2	29	0 (LSB)
29 / 2	14	1
14 / 2	7	0
7 / 2	3	1
3 / 2	1	1
1 / 2	0	1 (MSB)

58 / 2 | 29 | 0 (LSB) |

29 / 2 | 14 | 1 |

14 / 2 | 7 | 0 |

7 / 2 | 3 | 1 |

3 / 2 | 1 | 1 |

1 / 2 | 0 | 1 (MSB) |

- The integer part 58 in decimal is 111010 in binary.

Fractional Part Conversion (0.25):

- Repeatedly multiply the fractional part by 2, recording the integer part of the result.
- The binary digits are the integer parts, read from top to bottom.

Operation	Result	Carry
0.25 * 2	0.5	0
0.5 * 2	1.0	1

0.25 * 2 | 0.5 | 0 |

0.5 * 2 | 1.0 | 1 |

- The fractional part 0.25 in decimal is .01 in binary.

Combined Conversion:

- The binary equivalent of decimal 58.25 is 111010.01.

b) Decimal to Binary Conversion

Converting a decimal number to its binary equivalent involves two main operations:

1. **Integer Part Conversion:** This is achieved by repeatedly dividing the integer part of the decimal number by 2. The remainders, read from bottom to top, form the binary integer.

2. **Fractional Part Conversion:** This is done by repeatedly multiplying the fractional part of the decimal number by 2. The integer parts of the successive products, read from top to bottom, form the binary fraction.

Example 3

This example demonstrates the conversion of a mixed binary number (1101.11 $\blacksquare$) to its decimal equivalent.

Binary number: 1101.11 $\blacksquare$

Decimal equivalent: 13.75 $\blacksquare\blacksquare$

c) Binary Number to other Bases Conversion

Converting a binary number to bases other than decimal follows a distinct process.

This process is different from converting binary to decimal.

d) Binary to Hexa-Decimal Conversion

Binary and Hexadecimal number systems have bases of 2 and 16, respectively. A key relationship exists: $2^4 = 16$, meaning four binary bits (bits) are equivalent to one hexadecimal digit.

To convert a binary number to hexadecimal, follow these two steps:

1. **Grouping Bits:** Starting from the binary point, group the bits into sets of four. This grouping occurs on both sides of the binary point. If a group has fewer than four bits, pad it with zeros on the extreme sides to complete the group of four.
2. **Converting Groups:** Replace each group of four bits with its corresponding hexadecimal digit.

Binary to Decimal Conversion

To convert a binary number to its decimal equivalent, follow these steps:

1. **Multiply each bit** of the binary number by its corresponding **positional weight**.
2. **Sum all of these products** to get the final decimal value.

- **Positional weight:** The value assigned to each digit's position in a number system, typically a power of the base. For binary (base 2), these weights are powers of 2.

Example 4

This example demonstrates converting a binary number with a fractional part to its hexadecimal equivalent.

Binary to Hexadecimal Conversion (with fractional part):

1. **Group Bits:** Divide the binary number into groups of 4 bits, starting from the binary point and moving outwards.

- For groups that don't have 4 bits, pad with zeros on the extreme ends (left for the integer part, right for the fractional part) to complete the group of 4.
- Example: 101110.01101 becomes 0010 1110.0110 1000.

2. **Convert Groups:** Convert each 4-bit group into its corresponding hexadecimal digit. The relationship $2^4 = 16$ is key here.

- 0010 (binary) = 2 (hexadecimal)
- 1110 (binary) = E (hexadecimal)
- 0110 (binary) = 6 (hexadecimal)
- 1000 (binary) = 8 (hexadecimal)

3. **Combine:** Assemble the hexadecimal digits, maintaining the position relative to the hexadecimal point.

- 0010 1110.0110 1000 (binary) = 2E.68 (hexadecimal).

Hexadecimal to Binary Conversion:

This process is the reverse of binary-to-hexadecimal conversion. Each hexadecimal digit is directly represented by its equivalent 4-bit binary group.

Example 5

This example demonstrates converting the hexadecimal number 65.4C to its binary equivalent.

Each hexadecimal digit is represented by 4 bits:

- 6 (hex) = 0110 (binary)
- 5 (hex) = 0101 (binary)
- 4 (hex) = 0100 (binary)
- C (hex) = 1100 (binary)

Combining these, 65.4C₁₆ becomes 01100101.01001100₂.

Leading zeros in the integer part and trailing zeros in the fractional part can be removed without changing the value: 01100101.01001100₂ simplifies to 1100101.010011₂.

Therefore, the binary equivalent of 65.4C₁₆ is 1100101.010011₂.

2.1 Instruction Codes

An instruction code is a group of bits that tells the computer to perform a specific operation. A program is a set of instructions specifying operations, operands, and processing sequence.

An instruction code must specify:

- The operation to be performed.
- The registers where operands (data) are located.
- The register where the result should be stored.

Operation Code: A part of an instruction code that defines operations like add, subtract, multiply, shift, or complement. The number of bits for an operation code depends on the total number of available operations; at least 'n' bits are needed for 2^n operations.

2.0 Arithmetic Logical Unit

Instead of individual registers performing micro-operations, computer systems use an Arithmetic Logical Unit (ALU).

The ALU is a core component within the CPU responsible for all logical and mathematical operations.

Process:

1. Register contents are fed into the ALU's input.
2. The ALU performs the specified operation.
3. The result is transferred to a destination register.

2.2.1 Common Bus System

A common bus system provides an efficient way to transfer data between the registers and memory unit in a computer.

- The computer system includes registers, a memory unit, and a control unit.
- Paths are needed to move data between registers.
- The outputs of registers and memory are connected to this common bus.

2.2 Stored Program Organisation

In this organisation, instructions and data are stored in separate sections of memory.

A simple computer design uses a processor register, often called an Accumulator (ACC), and instruction codes. Each instruction code has two parts: the operation to perform and the memory address of the operand. The operation is executed using the memory operand and the current content of the ACC.

Load (LD)

Registers receive data from the common bus when their 'LD' (Load) input is enabled, with the data transfer occurring on the next clock pulse.

Instruction codes can specify operands in different ways:

- Immediate Operand: The second part of the instruction code directly specifies the operand's value.

- Direct Address: The second part of the instruction code specifies the memory address where the operand is located.
- Indirect Address: The second part of the instruction code specifies a memory address that contains the address of the actual operand.

2. Register Reference Instruction

Register Reference Instructions are a type of instruction recognized by a specific opcode pattern: '111' followed by a '0' in the leftmost bit of the instruction. The remaining 12 bits of the instruction are used to specify the exact operation the computer should perform.

Think of the opcode as a category label. For Register Reference Instructions, this label is '1110...'. The subsequent bits then act like specific commands within that category, telling the system precisely what to do with the registers.

1. Memory Reference Instruction

Memory Reference Instructions are a type of instruction code. They use 12 bits to specify a memory address and 1 bit to specify the addressing mode.

- Addressing Mode (I):
- I = 0: Direct Address (the instruction directly points to the memory location).
- I = 1: Indirect Address (the instruction points to a memory location that contains the actual address of the operand).

3. Input-Output Instruction

Input-Output (I/O) instructions are a type of instruction code that tells the computer to interact with external devices.

These instructions are identified by a specific operation code: '111'. Additionally, the leftmost bit of the instruction must be '1' to signify it's an I/O instruction.

The remaining 12 bits of the instruction are dedicated to specifying precisely which input or output operation should be performed.

2.2.2 Computer Instructions

The basic computer utilizes three distinct instruction code formats.

The Operation code (opcode) within an instruction is 3 bits long. The remaining 13 bits vary based on the specific opcode encountered.

a) Immediate Mode

In immediate mode, the operand itself is part of the instruction. Instead of an address field, the instruction contains the operand directly.

For instance, an instruction like 'ADD 7' means to add the value 7 (the operand) to the content of the accumulator. The number 7 is the operand in this case.

Format of Instruction

An instruction's format is represented by a rectangular box, detailing its constituent bits. The main fields are:

1. **Operation Code (Opcode) Field:** Specifies the operation to be performed.
2. **Address Field:** Designates a memory address or a register.
3. **Mode Field:** Dictates how the operand's effective address is determined.

Instructions can vary in length and the number of address fields, influenced by the computer's internal register organization.

2.2.3 Addressing Modes and Instruction Cycle

The **operation field** of an instruction dictates the action to be performed on data found in registers or main memory.

Addressing modes determine how operands are selected during program execution.

Purposes of addressing modes:

1. Provide programming versatility.
2. Reduce the number of bits needed for the addressing field within an instruction.

Types of Addressing Modes

Addressing modes define how the operand's location is specified within an instruction.

This section will discuss various addressing modes.

b) Register Mode

In Register Mode, the operand is directly stored within a CPU register.

The instruction itself contains the address of this specific register, indicating where the operand can be found for immediate use.

c) Register Indirect Mode

In Register Indirect Mode, an instruction specifies a register. The *contents* of this register are then used as the memory address of the operand.

Essentially, the register holds the operand's address, not the operand itself.

Advantages

Using specific addressing modes offers benefits:

- Faster access to operands.
- Instructions can be shorter, leading to quicker fetching of instructions.

Disadvantages

This system has a very limited address space, restricting the amount of memory it can directly access.

While using multiple registers can improve performance, it leads to more complex instructions.

d) Auto Increment/Decrement Mode

This addressing mode automatically adjusts the value of a register after (post-increment/decrement) or before (pre-increment/decrement) its value is used as an operand address.

Think of it like a bookmark that automatically moves forward or backward one page after you've read it, or before you start reading the next chapter.

e) `Direct Addressing Mode

In Direct Addressing Mode, the instruction itself contains the effective address of the operand.

- **Effective Address:** The actual location where the operand is stored.
- The instruction directly specifies this address (e.g., `ADD R1, 4000` where 4000 is the effective address).
- Requires only a single memory reference to access the data.
- No extra calculations are needed to determine the operand's location.

g) Displacement Addressing Mode

In displacement addressing mode, the effective address of an operand is calculated by adding the contents of an indexed register to the Address part of the instruction.

- **Effective Address:** The final memory location where the operand is found.
- **Indexed Register:** A CPU register whose value is used in address calculation.

f) Indirect Addressing Mode

In indirect addressing, the instruction's address field points to a memory location that *itself* contains the actual address of the operand. This requires an extra step (a second memory lookup) to find the operand, which slows down execution.

h) Relative Addressing Mode

Relative addressing is a form of Displacement addressing. The effective address (EA) is calculated by adding the address part of the instruction to the current content of the Program Counter (PC).

Formula:

$$EA = A + (PC)$$

This means the operand is located a certain number of cells away from the instruction currently being executed (pointed to by the PC).

j) Stack Addressing Mode

In Stack Addressing Mode, the operand is located at the top of the stack.

Example: An ADD instruction in this mode will first POP (remove) the top two items from the stack, then ADD them, and finally PUSH (add) the result back to the top of the stack.

i) Base Register Addressing Mode

This mode is a variation of Displacement addressing. The Effective Address (EA) is calculated as $EA = A + (R)$, where 'A' is a displacement value and 'R' is a register containing a base address pointer.

a) BCD Code or 8421 Code

Binary-Coded Decimal (BCD) is a coding scheme that represents decimal numbers using binary.

Also known as 8-4-2-1 code, this name reflects the positional weights of its four bits (8, 4, 2, 1), starting from the Most Significant Bit (MSB) with weight 8 (2^3) down to the Least Significant Bit (LSB) with weight 1 (2^0).

BCD is a weighted code, allowing arithmetic operations.

Example: The BCD bit sequence 0101 represents the decimal digit 5 ($08 + 14 + 02 + 11 = 5$).

Each decimal digit (0-9) is coded using four binary bits. The maximum decimal digit is 9.

BCD uses 4 bits per decimal digit. For example, decimal 35 is represented as 0011 0101 in BCD.

This requires more bits than straight binary (e.g., 35 in straight binary is 100011), but BCD is convenient for digital system input/output operations.

Forbidden Codes: Since only decimal digits 0-9 are coded, BCD uses only 10 of the 16 possible combinations of four bits. The unused combinations (1010, 1011, 1100, 1101, 1110, 1111) are known as forbidden codes or the forbidden group.

3.0 CODES AND THEIR CONVERSIONS

This section focuses on how information is represented and converted between different formats within computer systems.

BCD (8-4-2-1) Code:

- Represents decimal digits (0-9) using 4 binary bits.
- It's a weighted code, where each bit position has a specific weight (8, 4, 2, 1).
- Commonly used for Input/Output (I/O) operations.
- Has 'forbidden' unused codes (e.g., 1010 to 1111 in binary are not valid BCD representations for a single digit).

Conversions:

- The section previously detailed conversions between binary and decimal, including mixed numbers.
- It also covered binary to hexadecimal conversion by grouping bits (4 bits per hex digit) and the reverse process.
- Conversions from hexadecimal to binary were also demonstrated.

This section builds on number system conversions by introducing specific codes used in computing and reinforcing the understanding of data representation.

Introductions

Digital systems use binary signals but must process various data types (alphabetic, numeric, special characters). Information must be converted into a binary format for digital circuits.

A 'code' is a scheme that assigns a unique combination of 0s and 1s to each character or numeral.

Codes are broadly classified into five groups:

- Weighted Binary Codes
- Non-weighted Codes
- Error-detection Codes
- Error-correcting Codes
- Alphanumeric Codes

Different codes may be used within a single system for various operations, and code converter circuits are needed to transform data between code types.

A 'weighted binary code' assigns a specific weight to each position in a number. The decimal equivalent is found by multiplying each bit by its corresponding weight and summing the results.

b) 84-2-1 Code

The 84-2-1 code is a numerical code that assigns negative weights to its binary digits.

Key characteristics:

- It is a self-complementary code. This means the 9's complement of a decimal number can be found by simply inverting the bits (changing 1s to 0s and 0s to 1s) of its 84-2-1 code representation.
- Example: The decimal digit 3 is represented as 0101 ($0 \times 8 + 1 \times 4 + 0 \times (-2) + 1 \times (-1) = 3$). Its 9's complement is 6. Inverting the bits of 0101 yields 1010. Interpreting 1010 using the 84-2-1 weights gives $1 \times 8 + 0 \times 4 + 1 \times (-2) + 0 \times (-1) = 6$.

Example 3.2

This example demonstrates converting the decimal number 69.27 to its BCD equivalent.

- The decimal number is 69.27.
- Its BCD equivalent is 0110 1001 . 0010 0111.
- This shows the direct mapping of each decimal digit to its 4-bit BCD representation.

Example 3.1

Convert the decimal number 589 to its BCD equivalent.

The BCD equivalent is found by converting each decimal digit to its 4-bit BCD code.

$(5)_{10} = 0101$ BCD

$(8)_{10} = 1000$ BCD

$(9)_{10} = 1001$ BCD

Combining these gives:

$(589)_{10} = (0101\ 1000\ 1001)_{\text{BCD}}$

3.1.2 Nonweighted Codes

Nonweighted codes do not assign a fixed value to each position of a binary number.

Examples of nonweighted codes include:

- Excess-3 codes
- Gray codes

c) 2421 Code

The 2421 code is a weighted binary code where the weights assigned to the four binary digits are 2, 4, 2, and 1. This code is identical to BCD for decimal digits 0 through 4, but differs for digits 5 through 9.

Key characteristics:

- Weights: 2, 4, 2, 1.
- Self-complementary: The 9's complement of a decimal number is found by inverting its 2421 code bits (0s to 1s, 1s to 0s).
- Example: Decimal 4 is 0100. Decimal 7 is represented as 1101 ($21 + 41 + 20 + 11 = 7$).

A table showing the 2421 code alongside other codes (BCD, 8421, 84-2-1, Excess-3) for decimal digits 0-9 is provided for comparison.

a) Excess-3 Code

Excess-3 code is a nonweighted binary code used in some older computers to represent decimal digits.

To obtain the Excess-3 code for a decimal digit, 3 is added to the digit's value, and the result is then converted to its 4-bit binary equivalent. For example, the decimal digit 9 becomes 12 ($9+3$), which is 1100 in 4-bit binary.

Like 84-2-1 and 2421 codes, Excess-3 is self-complementary. This means the 9's complement of a decimal number can be found by inverting its Excess-3 code (0s become 1s, 1s become 0s). This property is useful for performing subtraction in digital systems.

Example 3.4

This example demonstrates converting the decimal number $(58.43)_{10}$ into its Excess-3 code.

Process:

1. Add 3 to each decimal digit.

- $5 + 3 = 8$
- $8 + 3 = 11$
- $4 + 3 = 7$
- $3 + 3 = 6$

2. Convert the resulting decimal numbers (8, 11, 7, 6) into their 4-bit binary equivalents.

- 8 -> 1000
- 11 -> 1011
- 7 -> 0111
- 6 -> 0110

Result:

The Excess-3 code for $(58.43)_{10}$ is (1000 1011.0111 0110).

Example 3.3

This example demonstrates converting a decimal number to its Excess-3 code.

To convert (367)₁₀ to Excess-3 code:

1. Take each decimal digit: 3, 6, 7.
2. Add 3 to each digit: $3+3=6$, $6+3=9$, $7+3=10$.
3. Convert each sum into its 4-bit binary equivalent: $6 = 0110$, $9 = 1001$, $10 = 1010$.

Hence, the Excess-3 code for (367)₁₀ is 0110 1001 1010.

Example 3.5. Convert (101011)₂ Solution

This example demonstrates converting a binary number to its Gray code equivalent.

Steps:

1. The Most Significant Bit (MSB) of the Gray code is identical to the MSB of the binary number.
2. Perform the Exclusive OR (ex-OR) operation between the MSB and the second bit of the binary number to obtain the second bit of the Gray code.
3. Perform the ex-OR operation between the second and third bits of the binary number to obtain the third bit of the Gray code.
4. Perform the ex-OR operation between the third and fourth bits of the binary number to obtain the fourth bit of the Gray code.

Result:

The Gray code for binary 101011 is 111110.

Binary to Gray Code Conversion Algorithm

Converting a binary number to its Gray code equivalent follows a simple algorithm:

1. **MSB Match:** The Most Significant Bit (MSB) of the Gray code is identical to the MSB of the binary number.
2. **Subsequent Bits:** Each subsequent bit in the Gray code is generated by performing an Exclusive OR (Ex-OR) operation between the corresponding bit and the bit immediately to its left (the MSB side) in the original binary number.
 - An Ex-OR operation results in '0' if the two bits are the same, and '1' if they are different.

This process continues for all bits of the binary number to produce the full Gray code representation.

b) Gray Code

Gray code is a type of **minimum change code**, meaning only one bit flips between successive numbers.

Key characteristics:

- **Minimum Change Property:** Adjacent values differ by only one bit. Imagine a rotary encoder where a small movement only changes one electrical contact.
- **Non-Weighted:** Unlike BCD, Gray code doesn't assign specific positional weights to its bits.
- **Applications:** Used in shaft encoders and some Analog-to-Digital converters.

Convert Gray Code to Binary

Converting Gray code to binary follows a step-by-step procedure:

1. **MSB Match:** The Most Significant Bit (MSB) of the binary number is identical to the MSB of the Gray code.
2. **Subsequent Bits:** Each subsequent binary bit is found by taking the Exclusive OR (Ex-OR) of the previously determined binary bit and the corresponding Gray code bit.
 - If the bits are the same, the result is 0.
 - If the bits are different, the result is 1.

This mechanism is applied sequentially to all lower-order bits to complete the conversion.

Example 3.6. Convert (564)₁₀ to Gray code Solution

The goal is to convert the decimal number 564 into its Gray code equivalent.

Step 1: Convert the decimal number 564 to its binary representation.

Binary representation of (564)₁₀ is 1000110100.

Example 3.7. Convert the Gray code 101101 into a binary number

To convert a Gray code to its binary equivalent:

1. **MSB Alignment:** The Most Significant Bit (MSB) of the binary number is identical to the MSB of the Gray code.
2. **Exclusive OR (XOR) Operations:** For subsequent bits:
 - XOR the MSB of the binary number with the next bit of the Gray code to find the next binary bit.
 - XOR the previously determined binary bit with the next Gray code bit to find the subsequent binary bit.
 - Continue this process until all bits are converted.

Applying this to Gray code 101101:

- MSB (Binary) = MSB (Gray) = 1.
- Next Binary Bit = 1 (MSB Binary) XOR 0 (2nd Gray) = 1.
- Next Binary Bit = 1 (Previous Binary) XOR 1 (3rd Gray) = 0.
- Next Binary Bit = 0 (Previous Binary) XOR 1 (4th Gray) = 1.
- Next Binary Bit = 1 (Previous Binary) XOR 0 (5th Gray) = 1.

- Next Binary Bit = 1 (Previous Binary) XOR 1 (6th Gray) = 0.

Resulting Binary Number: 110110.

a) Parity Bit Coding Technique

Binary data transmitted over mediums like wires or radio waves can be affected by noise, causing bit value changes (errors).

A parity bit is an extra bit added to a message to ensure the total count of 1s is either odd or even.

- **Odd Parity:** The parity bit is set so the total number of 1s (message + parity bit) is odd.
- **Even Parity:** The parity bit is set so the total number of 1s (message + parity bit) is even.

At the sending end, a parity generation circuit adds the parity bit to the message bits. At the receiving end, a parity check circuit verifies if the received message adheres to the adopted parity scheme.

An error is detected if the checked parity does not match the adopted one. The parity bit method can detect an odd number of errors (1, 3, 5, etc.). Even combinations of errors are undetectable as they do not change the overall parity.

The parity bit can be placed on the Most Significant Bit (MSB) or Least Significant Bit (LSB) side of the message. For other coding schemes, Check Sums are used.

3.1.3 Error-detection Codes

Error-detection codes are techniques used to identify if errors have occurred during binary data transmission.

- **Parity Bit:** A common method that adds an extra bit to a binary message to ensure either an odd or even number of 1s.
- **Sender:** Calculates the parity bit based on the message and appends it.
- **Receiver:** Recalculates the parity bit and compares it to the received parity bit. A mismatch indicates an error.

Analogy: Think of a parity bit like a simple checksum on a grocery receipt. You can quickly check if the total number of items (or in this case, '1' bits) matches what's expected, flagging a potential mistake if it doesn't.

b) Check Sums

Check Sums offer a more robust error detection than parity bits, especially for double errors.

Process:

1. At the transmitter, each word is added to a retained sum. This cumulative sum is called the Check Sum.
2. The final Check Sum is transmitted after all words.

3. The receiver performs the same addition process on the received words.
4. If the receiver's final sum equals the transmitted Check Sum, no error is detected.

a) Hamming Code

Hamming Code is a method developed by R. W. Hamming to systematically add parity bits to data for error detection and correction.

- **Hamming Distance (d):** The number of bits that differ between two code words. A greater Hamming distance allows for detection/correction of more errors.
- For single error detection, $d_{\min}$ must be at least 2.
- For single error correction, $d_{\min}$ must be at least 3.
- **Parity Bits (e.g., h1, h2, h4):** These bits are placed at positions corresponding to powers of two (1, 2, 4, 8, ...). Each parity bit checks a specific set of data bits based on the '1' positions in its binary representation.
- For example, h1 (LSB) checks positions with a '1' in the LSB.
- h2 (middle bit) checks positions with a '1' in the middle bit.
- h4 (MSB) checks positions with a '1' in the MSB.
- **Encoding (Example: 7-bit Hamming code for 4-bit data):** Data bits (b3, b2, b1) and parity bits (h4, h2, h1) are arranged. Parity bits are calculated to ensure even parity for their respective checked bit fields.
- **Decoding and Error Correction:** A non-zero parity word (e.g., a2, a1, a0) indicates an error at the bit position corresponding to the parity word's binary value. Complementing this bit corrects the error. A zero parity word signifies no error.

3.1.4 Error-correcting Codes

Error-correcting codes go beyond detection to fix errors. Hamming Code is a prime example, using parity bits and the concept of Hamming distance. Hamming distance measures the number of differing bits between two binary words; a smaller distance indicates higher similarity.

Hamming Code works by strategically placing parity bits within data. These parity bits are calculated based on specific subsets of the data bits. At the receiver, these parity bits are re-calculated and compared to the received parity bits. If they don't match, the pattern of mismatches (syndrome) pinpoints the exact bit that is in error, allowing it to be flipped and corrected.

This method is effective for detecting and correcting single-bit errors.

3.1.5 Alphanumeric Codes

Alphanumeric codes are binary codes used to represent a set of characters beyond just numbers.

These codes must handle:

- Ten decimal digits (0-9)
- The 26 letters of the alphabet (uppercase and lowercase)
- Special symbols (e.g., #, /, &, %)

To accommodate this larger character set (over 36 elements), a minimum of 6 bits are required. 6 bits allow for $2^6 = 64$ unique combinations, whereas 5 bits ($2^5 = 32$) would be insufficient.

3.1.7 EBCDIC

EBCDIC stands for "Extended Binary Coded Decimal Interchange Code."

It is an 8-bit alphanumeric code, with an additional ninth bit for parity.

EBCDIC is primarily used in IBM equipment and large computers for communicating alphanumeric data.

3.1.6 ASCII

ASCII (American Standard Code for Information Interchange) is a 7-bit coding scheme used in most microcomputers to represent characters.

Each character is stored using seven bits. Typically, this character is stored within a byte (8 bits), with one bit remaining unused. The unused bit can be used to extend ASCII, allowing for an additional 128 characters to be represented.

4.0 BOOLEAN ALGEBRA

Boolean algebra is a mathematical discipline used for the analysis and design of digital circuitry. It was proposed by George Boole and later applied to relay-switching circuits by Claude Shannon.

It uses logical variables that can be either 1 (TRUE) or 0 (FALSE).

Key Uses:

- **Analysis:** Economically describes the function of digital circuitry.
- **Design:** Develops simplified implementations of desired functions.

Basic logical operations are AND (.), OR (+), and NOT (overbar). Other operations include NOR, NAND, and XOR.

A truth table shows the outputs for all possible input combinations for a logic gate or circuit. It can be constructed for Boolean operators with two or more input variables.

4.1 Gates

Digital logic circuits are built from interconnected gates. A gate is an electronic circuit that performs a basic Boolean operation on its input signals to produce an output signal.

The six fundamental gates are AND, OR, NOT, NAND, NOR, and XOR. Each gate is defined by its graphical symbol, algebraic notation, and truth table.

- **AND Gate:** Output is 1 only if all inputs are 1. Algebraic: $A \cdot B$. Analogy: Like a series of light switches that must *all* be on for the main light to turn on.
- **OR Gate:** Output is 1 if any input is 1. Algebraic: $A + B$. Analogy: Like a light switch connected to multiple independent buttons; if *any* button is pressed, the light turns on.
- **NOT Gate (Inverter):** Output is the inverse of the input. Algebraic: A' . Analogy: A light switch that turns the light *off* when you flip it up and *on* when you flip it down.
- **NAND Gate:** Output is 0 only if all inputs are 1 (NOT AND). Algebraic: $(A \cdot B)'$.
- **NOR Gate:** Output is 1 only if all inputs are 0 (NOT OR). Algebraic: $(A + B)'$.
- **XOR Gate (Exclusive OR):** Output is 1 if an odd number of inputs are 1. Algebraic: $A \oplus B$.